

Aula 12 — Autenticação JWT no Blazor

Módulo: Blazor

Carga horária: 4 horas

Projetos base:

- **API:** Aula11.Api
- **Front:** Aula10.BlazorWasm (ou Blazor Server com HttpClient)

Objetivo da aula:

Compreender o **fluxo completo de autenticação JWT**, implementar **login com token**, **proteção de rotas**, **armazenamento seguro**, **refresh token** e **interceptação automática de requisições HTTP** no Blazor.

PARTE 1 — TEORIA (≈ 2 horas)

1 JWT Flow (JSON Web Token)

◆ O que é JWT?

Token **stateless**, assinado, usado para autenticação/autorização em APIs.

Estrutura:

HEADER.PAYLOAD.SIGNATURE

Fluxo completo:

Login → API valida credenciais → API gera Access Token (JWT) → Cliente armazena token → Cliente envia token no header
Authorization → API valida token a cada request

Header HTTP:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

2 Access Token vs Refresh Token

Token	Função	Tempo
Access Token	Acesso à API	Curto (5–15 min)
Refresh Token	Renovar acesso	Longo (dias)

👉 Boa prática:

Access Token curto + Refresh Token controlado no backend.

3 Armazenamento Seguro no Front

◆ Opções

- Blazor WASM
 - ProtectedLocalStorage
 - ProtectedSessionStorage
- **Nunca:** LocalStorage puro para dados sensíveis sem proteção

```
await localStorage.SetAsync("token", jwt);
```

4 Interceptors (HttpMessageHandler)

◆ Função

- Interceptar requisições HTTP
- Injetar JWT automaticamente
- Tratar 401 Unauthorized

- ✓ Centraliza segurança
 - ✓ Evita código duplicado
 - ✓ Padrão enterprise
-



PARTE 2 — PRÁTICA (≈ 2 horas)



Objetivo Prático

Implementar autenticação JWT end-to-end:

- API gera token
 - Blazor faz login
 - Token é armazenado com segurança
 - Rotas protegidas
 - Token enviado automaticamente em cada request
-



1 Configurando JWT na API



appsettings.json

```
"Jwt": { "Key": "chave-super-secreta-para-aula", "Issuer": "AulaApi", "Audience": "AulaBlazor", "ExpireMinutes": 10 }
```

Serviço de Token

 Services/TokenService.cs

```
using Microsoft.IdentityModel.Tokens; using System.IdentityModel.Tokens.Jwt; using System.Security.Claims; using System.Text;
namespace Aula11.Api.Services { public class TokenService { private readonly IConfiguration _config; public
TokenService(IConfiguration config) { _config = config; } public string GerarToken(string email) { var claims = new[] { new
Claim(ClaimTypes.Name, email), new Claim(ClaimTypes.Role, "Usuario") }; var key = new SymmetricSecurityKey(
Encoding.UTF8.GetBytes(_config["Jwt:Key"])); var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256); var token =
new JwtSecurityToken( issuer: _config["Jwt:Issuer"], audience: _config["Jwt:Audience"], claims: claims, expires:
DateTime.UtcNow.AddMinutes( int.Parse(_config["Jwt:ExpireMinutes"])), signingCredentials: creds); return new
JwtSecurityTokenHandler().WriteToken(token); } } }
```

Endpoint de Login

 Controllers/AuthController.cs

```
using Aula11.Api.Services; using Microsoft.AspNetCore.Mvc; namespace Aula11.Api.Controllers { [ApiController] [Route("api/auth")]
public class AuthController : ControllerBase { private readonly TokenService _tokenService; public AuthController(TokenService
tokenService) { _tokenService = tokenService; } [HttpPost("login")] public IActionResult Login(LoginDto dto) { // Simulação de
Login if (dto.Email == "admin@teste.com" && dto.Password == "123456") { var token = _tokenService.GerarToken(dto.Email); return
Ok(new { accessToken = token }); } return Unauthorized(); } } }
```

 LoginDto.cs

```
public record LoginDto(string Email, string Password);
```

2 Protegendo a API com JWT

 Program.cs (API)

```
builder.Services.AddAuthentication("Bearer").AddJwtBearer("Bearer", options => { options.TokenValidationParameters = new() {
    ValidateIssuer = true, ValidateAudience = true, ValidateLifetime = true, ValidateIssuerSigningKey = true, ValidIssuer =
    builder.Configuration["Jwt:Issuer"], ValidAudience = builder.Configuration["Jwt:Audience"], IssuerSigningKey = new
    SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"])) }; }); app.UseAuthentication();
app.UseAuthorization();
```

Proteger controller:

```
[Authorize] [HttpGet("seguro")] public IActionResult Seguro() { return Ok("Acesso autorizado"); }
```

3 Login no Blazor (WASM)

Serviço de Autenticação

 Services/AuthService.cs

```
using Microsoft.AspNetCore.Components.Server.ProtectedBrowserStorage; public class AuthService { private readonly HttpClient
_http; private readonly ProtectedLocalStorage _storage; public AuthService(HttpClient http, ProtectedLocalStorage storage) {
_http = http; _storage = storage; } public async Task<bool> LoginAsync(string email, string senha) { var response = await
_http.PostAsJsonAsync("api/auth/login", new { Email = email, Password = senha }); if (!response.IsSuccessStatusCode) return
false; var result = await response.Content.ReadFromJsonAsync<TokenResponse>(); await _storage.SetAsync("token",
result.AccessToken); return true; } public async Task<string?> GetTokenAsync() { var token = await _storage.GetAsync<string>
("token"); return token.Success ? token.Value : null; } public async Task LogoutAsync() { await _storage.DeleteAsync("token"); }
} public record TokenResponse(string AccessToken);
```

Registrar:

```
builder.Services.AddScoped<AuthService>(); builder.Services.AddProtectedBrowserStorage();
```

4 Interceptor HTTP (DelegatingHandler)

Handlers/JwtHandler.cs

```
using System.Net.Http.Headers; public class JwtHandler : DelegatingHandler { private readonly AuthService _authService; public
JwtHandler(AuthService authService) { _authService = authService; } protected override async Task<HttpResponseMessage> SendAsync(
HttpRequestMessage request, CancellationToken cancellationToken) { var token = await _authService.GetTokenAsync(); if
(!string.IsNullOrEmpty(token)) { request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", token); } return await
base.SendAsync(request, cancellationToken); } }
```

Registrar:

```
builder.Services.AddTransient<JwtHandler>(); builder.Services.AddHttpClient("Api") .AddHttpMessageHandler<JwtHandler>();
```

5 Proteção de Rotas no Blazor

App.razor

```
<CascadingAuthenticationState> <Router AppAssembly="@typeof(App).Assembly"> <Found Context="routeData"> <AuthorizeRouteView
RouteData="@routeData" DefaultLayout="@typeof(MainLayout)"> <NotAuthorized> <p>Acesso negado.</p> </NotAuthorized>
</AuthorizeRouteView> </Found> </Router> </CascadingAuthenticationState>
```

Página Protegida

```
@page "/admin" @attribute [Authorize] <h3>Área Administrativa</h3>
```

6 Refresh Token (Conceito Aplicado)

Fluxo recomendado:

401 Unauthorized → Cliente chama /refresh → Novo Access Token → Retry request original

📌 **Implementação completa** costuma envolver:

- Tabela de refresh tokens
- Revogação
- Device binding

(ideal para aula avançada ou pós).

✅ Resultado Esperado

- ✓ Login JWT funcional
 - ✓ Token armazenado com segurança
 - ✓ Rotas protegidas
 - ✓ API protegida
 - ✓ Interceptor automático
 - ✓ Arquitetura SPA profissional
-

📌 Atividade Avaliativa

👉 **Desafio prático:**

1. Implementar logout
 2. Criar role `Admin` no JWT
 3. Proteger rota por role
 4. Implementar expiração e redirecionamento
 5. Criar endpoint `/refresh`
-

🏁 Encerramento do Módulo 3 — Blazor

O aluno agora domina:

- Blazor Server
- Blazor WASM
- Comunicação com API
- Segurança JWT